

 [ravee360](#) / [Emotion_detection_using_CNN](#) Public

This project implements an Emotion Detection system using Convolutional Neural Networks (CNN) to classify facial expressions into seven categories: Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprise. It leverages transfer learning with pre-trained models like ResNet and VGGNet to improve accuracy and reduce training time.

 2 stars

 0 forks

 Branches

 Tags

 Activity

 Star

 Notifications

 Code

 Issues

 Pull requests

 Actions

 Projects

 Security

 Insights

 1 Branch

 0 Tags

 Go to file

 Go to file

 Code



 **ravee360** Update README.md 814caa2 · 6 months ago 

 .DS_Store	Add files via upload	7 months ago
 .gitattributes	added large files	7 months ago
 Custom_CNN_model...	added large files	7 months ago
 Final_Resnet50_Best_...	added large files	7 months ago
 README.md	Update README.md	6 months ago
 emotion_app.py	Add files via upload	7 months ago
 haarcascade_frontalfa...	Add files via upload	7 months ago
 requirements.txt	Add files via upload	7 months ago
 test.py	Add files via upload	7 months ago

 README 

Emotion Detection using CNN

This project implements a deep learning-based model for detecting human emotions from facial expressions using Convolutional Neural Networks (CNN). The model is trained on the [FER2013 Dataset](#), which contains facial images categorized into seven different emotions. The project explores multiple approaches, including building a CNN architecture from scratch, enhancing the model using image augmentation, and leveraging transfer learning techniques with VGGNet and ResNet50 to improve accuracy. Finally, the best-performing model is deployed for real-time emotion detection using OpenCV. Google colab link : <https://colab.research.google.com/drive/1ljuzbAVQINSL58dM1xDGo117GV-dlhTk?usp=sharing>

Table of Contents

- [Introduction](#)
- [Dataset](#)
- [Model Architectures](#)
 - [Model 1: Custom CNN](#)
 - [Model 2: Custom CNN with Image Augmentation](#)
 - [Model 3: Transfer Learning with VGGNet](#)
 - [Model 4: Transfer Learning with ResNet50](#)
- [Results](#)
- [Live Emotion Detection](#)
- [Technologies and Libraries Used](#)
- [Project Setup and Installation](#)
 - [Installation Guide](#)
 - [Running the Project](#)

Introduction

The goal of this project is to develop a machine learning solution that can identify human emotions such as anger, happiness, sadness, and others from facial expressions in images. Emotion detection has applications in various domains such as healthcare, entertainment, customer service, and human-computer interaction.

This project tackles the problem using Convolutional Neural Networks (CNN) to extract features from facial images and make predictions about the displayed emotion. Different strategies were employed to improve the model's performance, including transfer learning and image augmentation. The model was further integrated with OpenCV to enable real-time emotion detection from live video feeds.

Dataset

The [FER2013 dataset](#) was used to train and evaluate the models. This dataset contains 35,887 grayscale images of size 48x48 pixels, categorized into seven distinct emotions:

- Angry
- Disgust
- Fear
- Happy
- Sad
- Surprise
- Neutral

Each image is pre-processed and labeled for classification tasks, making it suitable for training deep learning models to recognize emotions.

Model Architectures

Model 1: Custom CNN

The first approach involves building a CNN model from scratch to classify emotions. The architecture consists of multiple convolutional layers followed by max-pooling, dropout, and fully connected layers.

- **Train Accuracy:** 63.11%
- **Validation Accuracy:** 55.52%

Despite achieving a reasonable accuracy, there was noticeable overfitting, as the validation accuracy lagged behind training accuracy.

Model 2: Custom CNN with Image Augmentation

To address the overfitting problem, image augmentation was applied to introduce variability in the training data. Techniques such as horizontal flipping, rotation, zooming, and shifting were employed.

- **Train Accuracy:** 57.64%
- **Validation Accuracy:** 59.28%

Image augmentation led to improved validation accuracy, demonstrating the model's better generalization to unseen data.

Model 3: Transfer Learning with VGGNet

In this model, VGGNet, a pre-trained deep learning model, was used as the base for transfer learning. Only the final layers were fine-tuned while leveraging VGGNet's pre-learned features. The model was trained for 50 epochs.

- **Train Accuracy:** 58.31%
- **Validation Accuracy:** 56.48%

Transfer learning with VGGNet helped the model converge faster but did not significantly improve accuracy.

Model 4: Transfer Learning with ResNet50

The best-performing model used ResNet50 for transfer learning. ResNet50's architecture allows for efficient feature extraction due to its skip connections, which mitigate the vanishing gradient problem.

- **Train Accuracy:** 61%+
- **Validation Accuracy:** 61%+

ResNet50 achieved the highest accuracy across all models, making it the optimal choice for this task.

Results

After experimenting with different architectures, the ResNet50-based model yielded the best results with an accuracy of over 61%. This model was then selected for deployment in real-time emotion detection applications.

Live Emotion Detection

The final trained model was integrated with OpenCV to enable real-time emotion detection using a webcam. The system processes each frame of the video, detects faces, and predicts the emotion of the person in real-time.

Features:

- Live capture from webcam
- Real-time emotion prediction
- Display of detected emotion on the screen

Technologies and Libraries Used

- **Programming Languages:** Python
- **Deep Learning Frameworks:** TensorFlow, Keras
- **Computer Vision Library:** OpenCV
- **Transfer Learning Models:** VGGNet, ResNet50
- **Other Libraries:** NumPy, Matplotlib, Seaborn, Pandas, Scikit-learn

Project Setup and Installation

Installation Guide

1. Clone the repository:

```
git clone https://github.com/ravee360/emotion-detection-cnn.git
cd emotion-detection-cnn
```



2. Create a virtual environment (optional but recommended):

```
python -m venv venv
source venv/bin/activate # On Windows, use `venv\Scripts\activate`
```



3. Install the required dependencies:

```
pip install -r requirements.txt
```



4. Download the FER2013 dataset from [Kaggle](https://www.kaggle.com/datasets/jessicali9530/fer2013) and place it in the appropriate directory.

Running the Project

Training the Model

To train the model from scratch, you can run the following command:

```
python train_model.py
```



Releases

No releases published

Packages

No packages published

Languages

● Python 100.0%